Code

# HIV-1 Nef & SERINC5: Replication and Extension of Rosa et al. 2015

Dhruvi Sheth 22294

April 17, 2026

# Background

Nef is a protein encoded by HIV-1. It dramatically increases how infectious new virus particles are, but *why* was unknown for 20 years. Rosa et al. (2015) solved this by looking at which host genes correlate with the Nef requirement across 15 cell lines. The answer turned out to be **SERINC5** - a host protein that normally blocks HIV, and which Nef disables.

We will: 1. **Replicate** their main result (SERINC5 expression correlates with infectivity ratio) 2. Apply proper **FDR correction** for multiple testing 3. Test **new biological hypotheses** beyond the paper

# Part 1 - Setup and Data Loading

Hide

```
# --- Load required libraries ---
# If not installed, run: install.packages(c("ggplot2","dplyr","tidyr","ggrepel","corrp
lot","ppcor"))
# For Ensembl gene name mapping: BiocManager::install(c("biomaRt","DESeq2"))

library(ggplot2)    # high-quality plots
library(dplyr)      # data wrangling (filter, mutate, arrange)
library(tidyr)      # reshaping data (pivot_wider, pivot_longer)
library(ggrepel)    # non-overlapping labels on scatter plots
library(corrplot)   # correlation matrix heatmaps
library(ppcor)      # partial and semi-partial correlations (Hypothesis 3)
```

Hide

```
# ============================================================
# STEP 1: Load infectivity data
# The infect.txt file has:
#   Column 1 = SRR accession (matches the .htseq filename)
#   Column 2 = Nef+/Nef- infectivity ratio
#   Column 3 = cell line name
# A HIGH ratio means Nef is needed a lot → SERINC5 likely high
# A LOW  ratio means Nef doesn't do much → SERINC5 likely low
# ============================================================

infect <- read.table("../data/infect.txt",
                     header = FALSE,
                     col.names = c("SRR", "infectivity_ratio", "cell_line"),
                     sep = "\t")

# Sort by infectivity ratio so we can see high vs low Nef-dependent cells
infect <- infect[order(infect$infectivity_ratio), ]
rownames(infect) <- NULL

# ============================================================
# CREATE GROUP VARIABLE (needed for plots)
# ============================================================

infect$group <- ifelse(infect$infectivity_ratio >= median(infect$infectivity_ratio),
                       "High Nef-dependent",
                       "Low Nef-dependent")

infect$group <- factor(infect$group,
                       levels = c("Low Nef-dependent", "High Nef-dependent"))
cat("Infectivity data loaded.\n")
```

```
Infectivity data loaded.
```

Hide

```
print(infect)
```

```
          SRR infectivity_ratio   cell_line              group
1  SRR2166626              0.79      HT1080  Low Nef-dependent
2  SRR2166634              0.98   CEM X 174  Low Nef-dependent
3  SRR2166632              1.02       IMR90  Low Nef-dependent
4  SRR2166624              1.19         MT4  Low Nef-dependent
5  SRR2166629              1.27       DAUDI  Low Nef-dependent
6  SRR2166635              1.31        WI38  Low Nef-dependent
7  SRR2166630              1.38       C8166  Low Nef-dependent
8  SRR2166627              1.57        RAJI High Nef-dependent
9  SRR2166625              9.96        HSB2 High Nef-dependent
10 SRR2166628             12.85      CEM SS High Nef-dependent
11 SRR2166633             13.92   CEM A 301 High Nef-dependent
12 SRR2166631             14.44       RAMOS High Nef-dependent
13 SRR2166637             15.78        bl41 High Nef-dependent
14 SRR2166636             32.35 JURKAT E6.1 High Nef-dependent
15 SRR2166638             34.04 Jurkat tag1 High Nef-dependent
```

Hide

```
# ============================================================
# STEP 2: Load all 15 RNA-seq files into one big count matrix
# Each .htseq file has: gene_ID (Ensembl) and raw read count
# The last 5 lines start with "__" (HTSeq summary lines)-we remove those
# ============================================================

data_dir <- "../data/"
htseq_files <- list.files(data_dir, pattern = "\\.htseq$", full.names = TRUE)

# Read one file first to get gene IDs (all files have same genes in same order)
first_file  <- read.table(htseq_files[1], header = FALSE,
                          col.names = c("gene_id", "count"), sep = "\t")

# Remove the HTSeq summary lines (they start with "__")
first_file <- first_file[!grepl("^__", first_file$gene_id), ]

# Build the count matrix: rows = genes, columns = samples
count_matrix <- data.frame(gene_id = first_file$gene_id)

# Initialize empty list
count_list <- list()

for (f in htseq_files) {
  sample_name <- sub("\\.htseq$", "", basename(f))   # extract SRR id
  tmp <- read.table(f, header = FALSE,
                    col.names = c("gene_id", "count"), sep = "\t")
  tmp <- tmp[!grepl("^__", tmp$gene_id), ]           # remove summary rows
# Rename count column to sample name
  colnames(tmp)[2] <- sample_name

  # Store in list
  count_list[[sample_name]] <- tmp
}

# Merge all dataframes by gene_id
count_matrix <- Reduce(function(x, y) merge(x, y, by = "gene_id"), count_list)

# Set rownames
rownames(count_matrix) <- count_matrix$gene_id
count_matrix <- count_matrix[, -1]

# Replace NA with 0
count_matrix[is.na(count_matrix)] <- 0

cat("Final matrix dimensions:\n")
```

```
Final matrix dimensions:
```

Hide

```
dim(count_matrix)
```

```
[1] 58302    15
```

Hide

```
# After merging all files
count_matrix <- Reduce(function(x, y) merge(x, y, by = "gene_id"), count_list)

# Set rownames
rownames(count_matrix) <- count_matrix$gene_id

# Remove gene_id column
count_matrix <- count_matrix[, -1]

# IMPORTANT: restore column names from list
colnames(count_matrix) <- names(count_list)

# Replace NA with 0
count_matrix[is.na(count_matrix)] <- 0

# Check
colnames(count_matrix)
```

```
 [1] "SRR2166624" "SRR2166625" "SRR2166626" "SRR2166627" "SRR2166628" "SRR2166629" "SR
R2166630" "SRR2166631" "SRR2166632" "SRR2166633" "SRR2166634" "SRR2166635" "SRR2166636
" "SRR2166637" "SRR2166638"
```

Hide

```
# ============================================================
# STEP 4: Normalize raw counts to RPM
# ============================================================

# count_matrix already has:
# rows = genes
# columns = samples
# rownames = gene_ids

# Normalize
library_sizes <- colSums(count_matrix)
rpm_matrix <- sweep(count_matrix, 2, library_sizes, FUN = "/") * 1e6

# Log transform
expr_log <- log2(rpm_matrix + 1)

# Check again
colnames(expr_log)
```

```
 [1] "SRR2166624" "SRR2166625" "SRR2166626" "SRR2166627" "SRR2166628" "SRR2166629" "SR
R2166630" "SRR2166631" "SRR2166632" "SRR2166633" "SRR2166634" "SRR2166635" "SRR2166636
" "SRR2166637" "SRR2166638"
```

Hide

```
# Reorder correctly
expr_log <- expr_log[, infect$SRR]

# Final check
all(colnames(expr_log) == infect$SRR)
```

```
[1] TRUE
```

Hide

```
# Check one gene (use correct gene name later)
head(rpm_matrix)
```

| | SRR2166... | SRR2166625 | SRR2166... | SRR2166... | SRR2166... | SRR2166... |
|---|---|---|---|---|---|---|
| | <dbl> | <dbl> | <dbl> | <dbl> | <dbl> | <dbl> |
| ENSG00000000003 | 0.0000000 | 4.81296698 | 9.9611068 | 0.00000 | 0.0000000 | 0.00000 |
| ENSG00000000005 | 0.0000000 | 0.07520261 | 0.0000000 | 0.00000 | 0.0000000 | 0.00000 |
| ENSG00000000419 | 40.0297043 | 53.16824456 | 75.0590441 | 49.63045 | 57.0600174 | 44.76083 |
| ENSG00000000457 | 48.1104670 | 48.35527758 | 23.1490510 | 33.75765 | 39.6025616 | 50.14480 |
| ENSG00000000460 | 66.2173613 | 62.11735502 | 20.0625109 | 36.21681 | 65.3038159 | 69.04147 |
| ENSG00000000938 | 0.6733969 | 0.60162087 | 0.1402973 | 166.38496 | 0.2424647 | 125.41478 |

6 rows | 1-8 of 15 columns

Hide

```
# ============================================================
# STEP 4: Log transformation
# Needed because RNA-seq counts are highly skewed
# ============================================================

expr_log <- log2(count_matrix + 1)

# Check
summary(expr_log[1:5, 1:5])
```

```
   SRR2166624      SRR2166625      SRR2166626      SRR2166627      SRR2166628
 Min.   :0.000   Min.   :1.000   Min.   :0.000   Min.   :0.000   Min.   :0.000
 1st Qu.:0.000   1st Qu.:6.022   1st Qu.:6.170   1st Qu.:0.000   1st Qu.:0.000
 Median :9.066   Median :9.331   Median :7.170   Median :9.241   Median :8.940
 Mean   :5.638   Mean   :7.103   Mean   :5.956   Mean   :5.676   Mean   :5.613
 3rd Qu.:9.331   3rd Qu.:9.468   3rd Qu.:7.375   3rd Qu.:9.342   3rd Qu.:9.466
 Max.   :9.791   Max.   :9.692   Max.   :9.066   Max.   :9.796   Max.   :9.660
```

Hide

```
# ============================================================
# STEP 5: Match sample names (IMPORTANT STEP)
# RNA-seq column names = SRR IDs
# Infect file has SRR IDs → align both datasets
# ============================================================

# Reorder RNA matrix columns to match infect file
expr_log <- expr_log[, infect$SRR]

# Check alignment
colnames(expr_log)
```

```
 [1] "SRR2166626" "SRR2166634" "SRR2166632" "SRR2166624" "SRR2166629" "SRR2166635" "SR
R2166630" "SRR2166627" "SRR2166625" "SRR2166628" "SRR2166633" "SRR2166631" "SRR2166637
" "SRR2166636" "SRR2166638"
```

Hide

```
infect$SRR
```

```
 [1] "SRR2166626" "SRR2166634" "SRR2166632" "SRR2166624" "SRR2166629" "SRR2166635" "SR
R2166630" "SRR2166627" "SRR2166625" "SRR2166628" "SRR2166633" "SRR2166631" "SRR2166637
" "SRR2166636" "SRR2166638"
```

Hide

```
colnames(count_matrix)
```

```
 [1] "SRR2166624" "SRR2166625" "SRR2166626" "SRR2166627" "SRR2166628" "SRR2166629" "SR
R2166630" "SRR2166631" "SRR2166632" "SRR2166633" "SRR2166634" "SRR2166635" "SRR2166636
" "SRR2166637" "SRR2166638"
```

Hide

```
dim(count_matrix)
```

```
[1] 58302    15
```

Hide

```
colnames(rpm_matrix)
```

```
 [1] "SRR2166624" "SRR2166625" "SRR2166626" "SRR2166627" "SRR2166628" "SRR2166629" "SR
R2166630" "SRR2166631" "SRR2166632" "SRR2166633" "SRR2166634" "SRR2166635" "SRR2166636
" "SRR2166637" "SRR2166638"
```

Hide

```
# ============================================================
# STEP 6: Extract SERINC genes
# (Check if names match exactly in your dataset)
# ============================================================

# Extract using Ensembl IDs
serinc5 <- as.numeric(expr_log["ENSG00000197217", ])
serinc3 <- as.numeric(expr_log["ENSG00000172270", ])

# Check values
serinc5
```

```
 [1] 10.293472  6.930737 12.098690 10.793603 10.695228 11.577901  9.357552  9.738092
9.930737 10.355351  5.000000 10.887982  8.823367 11.689998 11.003518
```

Hide

```
serinc3
```

```
 [1] 11.432542  8.857981 12.493105 10.570804  9.977280 12.160187 10.276124 11.563196 1
1.314583 11.576957  5.807355  9.355351 11.265615 11.220378 11.197217
```

Hide

```
"ENSG00000197217" %in% rownames(expr_log)
```

```
[1] TRUE
```

Hide

```
"ENSG00000172270" %in% rownames(expr_log)
```

```
[1] TRUE
```

# Part 2-Descriptive Statistics

Hide

```
# ============================================================
# STEP 4: Descriptive statistics on infectivity ratio
# This gives us the "shape" and spread of our main outcome variable
# ============================================================

cat("=== Descriptive Statistics: Infectivity Ratio ===\n")
```

```
=== Descriptive Statistics: Infectivity Ratio ===
```

Hide

```
cat("Mean:   ", mean(infect$infectivity_ratio), "\n")
```

```
Mean:    9.523333
```

Hide

```
cat("Median: ", median(infect$infectivity_ratio), "\n")
```

```
Median:  1.57
```

Hide

```
cat("SD:     ", sd(infect$infectivity_ratio), "\n")
```

```
SD:      11.25099
```

Hide

```
cat("Min:    ", min(infect$infectivity_ratio), "\n")
```

```
Min:     0.79
```

Hide

```
cat("Max:    ", max(infect$infectivity_ratio), "\n")
```

```
Max:     34.04
```

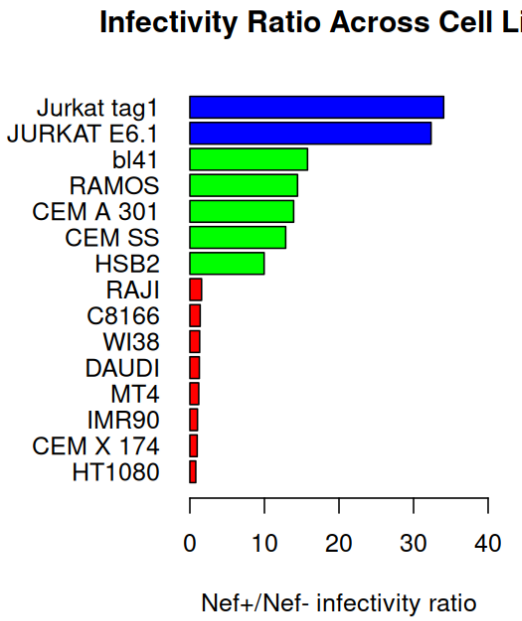
Hide

```
# If Mean > Median → right-skewed (as taught in class)
if (mean(infect$infectivity_ratio) > median(infect$infectivity_ratio)) {
  cat("\nMean > Median → Distribution is RIGHT-SKEWED\n")
} else {
  cat("\nMean < Median → Distribution is LEFT-SKEWED\n")
}
```

```


Mean > Median → Distribution is RIGHT-SKEWED
```

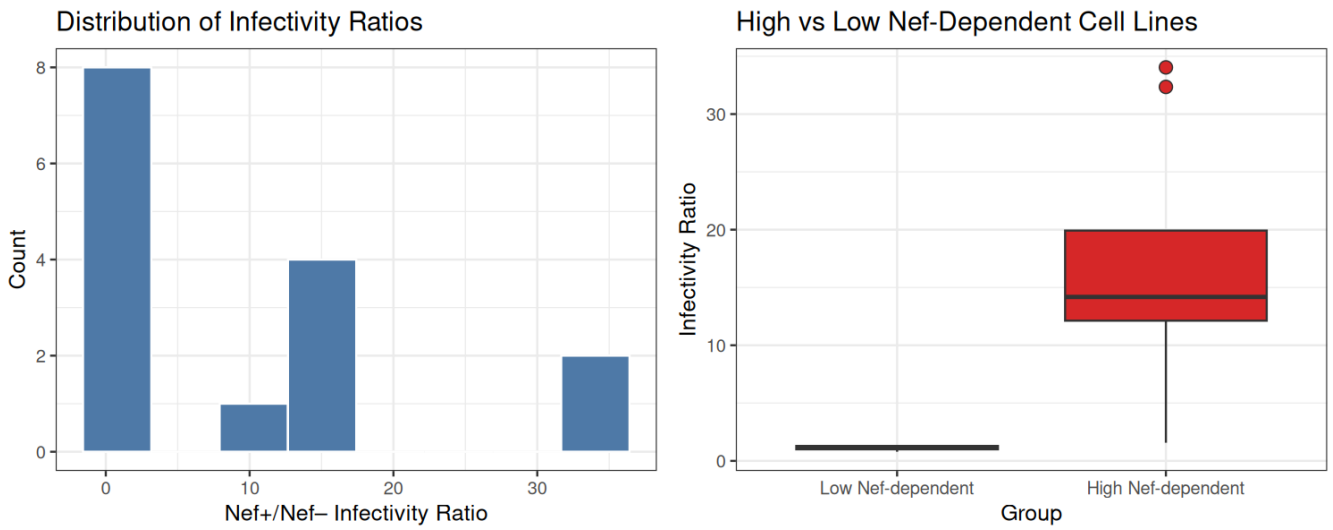
Hide

```
# Bar plot of infectivity ratio per cell line
# This replicates Figure 1a from Rosa et al. 2015

# ==============================================================
# STEP 3: Barplot of infectivity (replicating paper idea)
# ==============================================================

par(mar = c(5, 5, 4, 2))
par(pin = c(2, 2.8))

# assign colors
colors <- ifelse(infect$infectivity_ratio >= 30, "blue",
          ifelse(infect$infectivity_ratio >= 9, "green",
          ifelse(infect$infectivity_ratio <= 2, "red", "grey")))

barplot(infect$infectivity_ratio,
        names.arg = infect$cell_line,
        col = colors,
        horiz = TRUE,
        las = 2,
        xaxt = "n",
        xlim = c(0, 40),
        xlab = "Nef+/Nef- infectivity ratio",
        main = "Infectivity Ratio Across Cell Lines")

axis(1, at = seq(0, 40, by = 10))
```




Hide

```
# Visualise distribution: histogram + boxplot side by side

p1 <- ggplot(infect, aes(x = infectivity_ratio)) +
  geom_histogram(bins = 8, fill = "#4e79a7", color = "white") +
  labs(title = "Distribution of Infectivity Ratios",
       x = "Nef+/Nef– Infectivity Ratio", y = "Count") +
  theme_bw(base_size = 12)

p2 <- ggplot(infect, aes(y = infectivity_ratio, x = group, fill = group)) +
  geom_boxplot(outlier.shape = 21, outlier.size = 3) +
  scale_fill_manual(values = c("High Nef-dependent" = "#d62728",
                               "Low Nef-dependent"  = "#1f77b4")) +
  labs(title = "High vs Low Nef-Dependent Cell Lines",
       x = "Group", y = "Infectivity Ratio") +
  theme_bw(base_size = 12) +
  theme(legend.position = "none")

gridExtra::grid.arrange(p1, p2, ncol = 2)
```



Hide

```
# ==============================================================
# STEP 5: Test normality of infectivity ratio
# As taught in class: use histogram + QQ plot + Shapiro-Wilk test
# H0 (null hypothesis): data is normally distributed
# If p > 0.05 → accept H0 (normal); if p < 0.05 → reject H0 (not normal)
# ==============================================================

shapiro_infect <- shapiro.test(infect$infectivity_ratio)
cat("Shapiro-Wilk test on infectivity ratio:\n")
```

```
Shapiro-Wilk test on infectivity ratio:
```

Hide

```
cat("W =", shapiro_infect$statistic, "\n")
```

```
W = 0.7630091
```

Hide

```
cat("p-value =", shapiro_infect$p.value, "\n")
```

```
p-value = 0.001280347
```

Hide

```
if (shapiro_infect$p.value < 0.05) {
  cat("→ p < 0.05: REJECT H0. Infectivity ratio is NOT normally distributed.\n")
  cat("→ We will use LOG transformation before correlation analysis.\n")
} else {
  cat("→ p > 0.05: FAIL TO REJECT H0. Infectivity ratio appears normally distributed.
\n")
}
```

```
→ p < 0.05: REJECT H0. Infectivity ratio is NOT normally distributed.
→ We will use LOG transformation before correlation analysis.
```

Hide

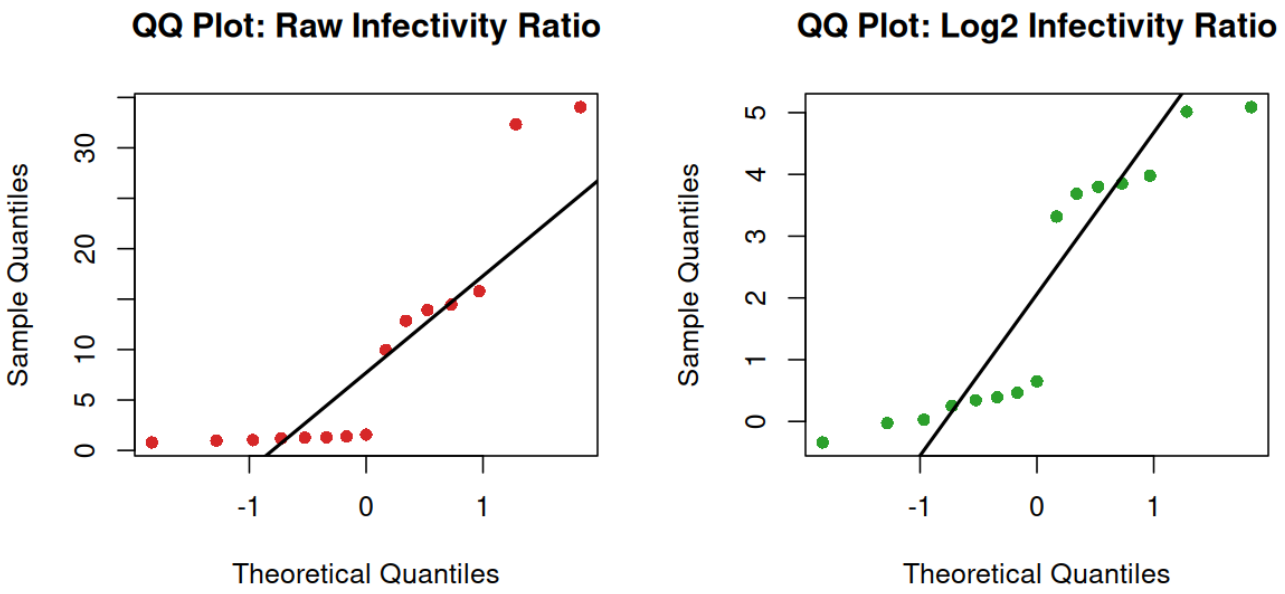
```
# QQ plot for infectivity ratio (raw and log-transformed)
par(mfrow = c(1, 2))

# Raw data QQ plot
qqnorm(infect$infectivity_ratio, main = "QQ Plot: Raw Infectivity Ratio",
       pch = 16, col = "#d62728")
qqline(infect$infectivity_ratio, col = "black", lwd = 2)
```

Hide

```
# Log-transformed QQ plot
qqnorm(log2(infect$infectivity_ratio), main = "QQ Plot: Log2 Infectivity Ratio",
       pch = 16, col = "#2ca02c")
qqline(log2(infect$infectivity_ratio), col = "black", lwd = 2)
```

Hide

```
par(mfrow = c(1, 1))
```




Hide

```
# Log2 transformation makes the skewed infectivity ratio more symmetric
# This is standard in genomics: log2 is preferred (fold-change interpretable)
infect$log2_infectivity <- log2(infect$infectivity_ratio)

shapiro_log <- shapiro.test(infect$log2_infectivity)
cat("Shapiro-Wilk after log2 transformation:\n")
```

```
Shapiro-Wilk after log2 transformation:
```

Hide

```
cat("W =", shapiro_log$statistic, ", p =", shapiro_log$p.value, "\n")
```

```
W = 0.8209421 , p = 0.006914355
```

# Part 3-Correlation Analysis (Replicating the Paper)

The core analysis: for every gene, compute its Pearson correlation with the infectivity ratio. Then apply FDR correction.

Hide

```
# ============================================================
# STEP 6: Match sample order in RPM matrix to infect dataframe
# The rpm_matrix columns are SRR IDs-reorder to match infect$SRR
# ============================================================

# Make sure SRR IDs match between expression data and infectivity data
shared_srr <- intersect(infect$SRR, colnames(rpm_matrix))
cat("Samples in both infectivity and expression data:", length(shared_srr), "\n")
```

```
Samples in both infectivity and expression data: 15
```

Hide

```
# Reorder RPM matrix columns to match the order in infect table
rpm_aligned <- rpm_matrix[, shared_srr]
infect_aligned <- infect[match(shared_srr, infect$SRR), ]

# Sanity check: columns of RPM matrix should now match rows of infect
cat("SRR order matches:", all(colnames(rpm_aligned) == infect_aligned$SRR), "\n")
```

```
SRR order matches: TRUE
```

Hide

```
# ============================================================
# STEP 7: Compute Pearson correlation of every gene with infectivity
#
# Why Pearson? As taught in class, Pearson correlation measures the
# LINEAR relationship between two continuous variables. Here:
#   x = gene expression (RPM) across 15 cell lines
#   y = Nef+/Nef- infectivity ratio across same 15 cell lines
#
# We use log2(infectivity) since we showed it is more normally distributed.
#
# For ~58,000 genes, we do this efficiently using apply()
# ============================================================

log2_infect_vec <- infect_aligned$log2_infectivity

# For each gene (row), compute Pearson correlation with log2 infectivity
# cor.test() gives r (correlation coefficient) and p-value
# We use sapply to loop over all genes efficiently

cat("Computing Pearson correlation for all genes... (this may take a moment)\n")
```

```
Computing Pearson correlation for all genes... (this may take a moment)
```

Hide

```
cor_results <- t(apply(rpm_aligned, 1, function(gene_expr) {
  # Filter: skip genes with zero expression in all samples (can't correlate)
  if (sum(gene_expr) == 0) return(c(r = NA, p = NA))

  result <- tryCatch(
    cor.test(as.numeric(gene_expr), log2_infect_vec, method = "pearson"),
    error = function(e) list(estimate = NA, p.value = NA)
  )
  c(r = as.numeric(result$estimate), p = result$p.value)
}))

cor_df <- as.data.frame(cor_results)
cor_df$gene_id <- rownames(cor_df)

# Remove genes where correlation could not be computed (all zeros)
cor_df <- cor_df[!is.na(cor_df$r), ]

cat("Genes with computable correlations:", nrow(cor_df), "\n")
```

```
Genes with computable correlations: 45685
```

Hide

```
cat("Genes removed (all zero expression):", nrow(rpm_aligned) - nrow(cor_df), "\n")
```

```
Genes removed (all zero expression): 12617
```

Hide

```
# ============================================================
# STEP 8: FDR correction for multiple testing
#
# We tested ~20,000+ genes. At alpha = 0.05, we expect
# 0.05 × 20000 = 1000 FALSE POSITIVES just by chance!
# (as explained in class by Dhruvi Sheth, March 11)
#
# Strategy: Apply Benjamini-Hochberg (BH) FDR correction
# This gives us an "adjusted p-value" (q-value)
#
# IMPORTANT DECISION (as the project asks):
# Should we correct on ALL genes or a SUBSET?
#
# Biologically informed approach: The paper used only genes with
# detectable expression (RPM > 0 in at least some samples).
# We further filter to genes expressed in ≥ 8/15 samples with RPM > 1
# This is valid because genes not expressed in these cells cannot
# biologically explain the infectivity differences.
# ============================================================

# Filter to expressed genes (RPM > 1 in at least 8 of 15 samples)
expressed_genes <- rownames(rpm_aligned)[
  rowSums(rpm_aligned > 1) >= 8
]
cat("Expressed genes (RPM > 1 in ≥ 8 samples):", length(expressed_genes), "\n")
```

```
Expressed genes (RPM > 1 in ≥ 8 samples): 15365
```

Hide

```
# Subset correlation results to expressed genes
cor_expressed <- cor_df[cor_df$gene_id %in% expressed_genes, ]

# Apply BH FDR correction on these expressed genes
cor_expressed$p_adj_BH <- p.adjust(cor_expressed$p, method = "BH")

# Also apply stringent Bonferroni for comparison
cor_expressed$p_adj_bonferroni <- p.adjust(cor_expressed$p, method = "bonferroni")

# Sort by correlation coefficient (descending = most positively correlated first)
cor_expressed <- cor_expressed[order(cor_expressed$r, decreasing = TRUE), ]

# How many pass different thresholds?
cat("\n--- Significant genes after FDR correction ---\n")
```

```


--- Significant genes after FDR correction ---
```

Hide

```
cat("Raw p < 0.05:              ", sum(cor_expressed$p < 0.05, na.rm=TRUE), "\n")
```

```
Raw p < 0.05:               2633
```

Hide

```
cat("BH FDR (q < 0.05):        ", sum(cor_expressed$p_adj_BH < 0.05, na.rm=TRUE),
"\n")
```

```
BH FDR (q < 0.05):         15
```

Hide

```
cat("Bonferroni (p_adj < 0.05):", sum(cor_expressed$p_adj_bonferroni < 0.05, na.rm=TRU
E), "\n")
```

```
Bonferroni (p_adj < 0.05): 2
```

Hide

```
# ============================================================
# STEP 9: Add human-readable gene names (HGNC symbols)
# As noted in class (class_12.md): Ensembl IDs need to be converted
# to HGNC gene names. We manually annotate key genes here,
# then use biomaRt for the full mapping.
# ============================================================

# ============================================================
# CLEAN SERINC MAPPING (FINAL FIX)
# ============================================================

serinc_ids <- c(
  SERINC5 = "ENSG00000197217",
  SERINC3 = "ENSG00000100564",
  SERINC1 = "ENSG00000197121",
  SERINC2 = "ENSG00000149403",
  SERINC4 = "ENSG00000197256"
)

serinc5_id <- serinc_ids["SERINC5"]
serinc3_id <- serinc_ids["SERINC3"]

# ============================================================
# CHECK MATCHING IN DATA
# ============================================================

serinc_results <- cor_expressed[
  cor_expressed$gene_id %in% unname(serinc_ids),
  c("gene_id","r","p","p_adj_BH")
]

# attach names properly
serinc_results$gene_name <- names(serinc_ids)[
  match(serinc_results$gene_id, serinc_ids)
]

print(serinc_results)
```

```
                        gene_id          r          p  p_adj_BH gene_name
ENSG00000100564 ENSG00000100564  0.6071217 0.01638607 0.1957125   SERINC3
ENSG00000197217 ENSG00000197217  0.0342220 0.90363009 0.9635370   SERINC5
ENSG00000197121 ENSG00000197121 -0.2003571 0.47400927 0.7147818   SERINC1
ENSG00000197256 ENSG00000197256 -0.3068017 0.26603137 0.5483730   SERINC4
```

Hide

```
# Add a column for gene name (gene_id for most, readable name for SERINC genes)
cor_expressed$gene_name <- ifelse(
  cor_expressed$gene_id %in% names(serinc_ids),
  serinc_ids[cor_expressed$gene_id],
  cor_expressed$gene_id
)

# Show SERINC gene results
cat("\n=== SERINC family correlations with infectivity ===\n")
```

```
=== SERINC family correlations with infectivity ===
```

Hide

```
serinc_results <- cor_expressed[cor_expressed$gene_id %in% names(serinc_ids),
                                c("gene_name","r","p","p_adj_BH")]
serinc_results <- serinc_results[order(serinc_results$r, decreasing = TRUE), ]
print(serinc_results)
```

```
[1] gene_name r         p         p_adj_BH
<0 rows> (or 0-length row.names)
```

Hide

```
rpm_aligned[serinc5_id, ]
```

| | SRR2166... <dbl> | SRR2166... <dbl> | SRR2166... <dbl> | SRR2166... <dbl> | SRR2166... <dbl> | SRR2166... <dbl> | S |
|---|---|---|---|---|---|---|---|
| ENSG00000197217 | 175.9328 | 47.40614 | 161.3897 | 132.734 | 174.9262 | 131.966 | |

1 row | 1-8 of 15 columns

Hide

```
grep("97217", rownames(count_matrix), value = TRUE)
```

```
[1] "ENSG00000197217"
```

Hide

```
rpm_aligned["ENSG00000197217", ]
```

| | SRR2166... <dbl> | SRR2166... <dbl> | SRR2166... <dbl> | SRR2166... <dbl> | SRR2166... <dbl> | SRR2166... <dbl> | S |
|---|---|---|---|---|---|---|---|
| ENSG00000197217 | 175.9328 | 47.40614 | 161.3897 | 132.734 | 174.9262 | 131.966 | |

1 row | 1-8 of 15 columns

Hide

```
count_matrix["ENSG00000197217", ]
```

| | SRR2166... <int> | SRR2166... <int> | SRR2166... <int> | SRR2166... <int> | SRR2166... <int> | SRR2166... <int> | S |
|---|---|---|---|---|---|---|---|
| ENSG00000197217 | 1774 | 975 | 1254 | 853 | 1309 | 1657 | |

1 row | 1-8 of 15 columns

Hide

```
rpm_aligned[serinc5_id, ]
```

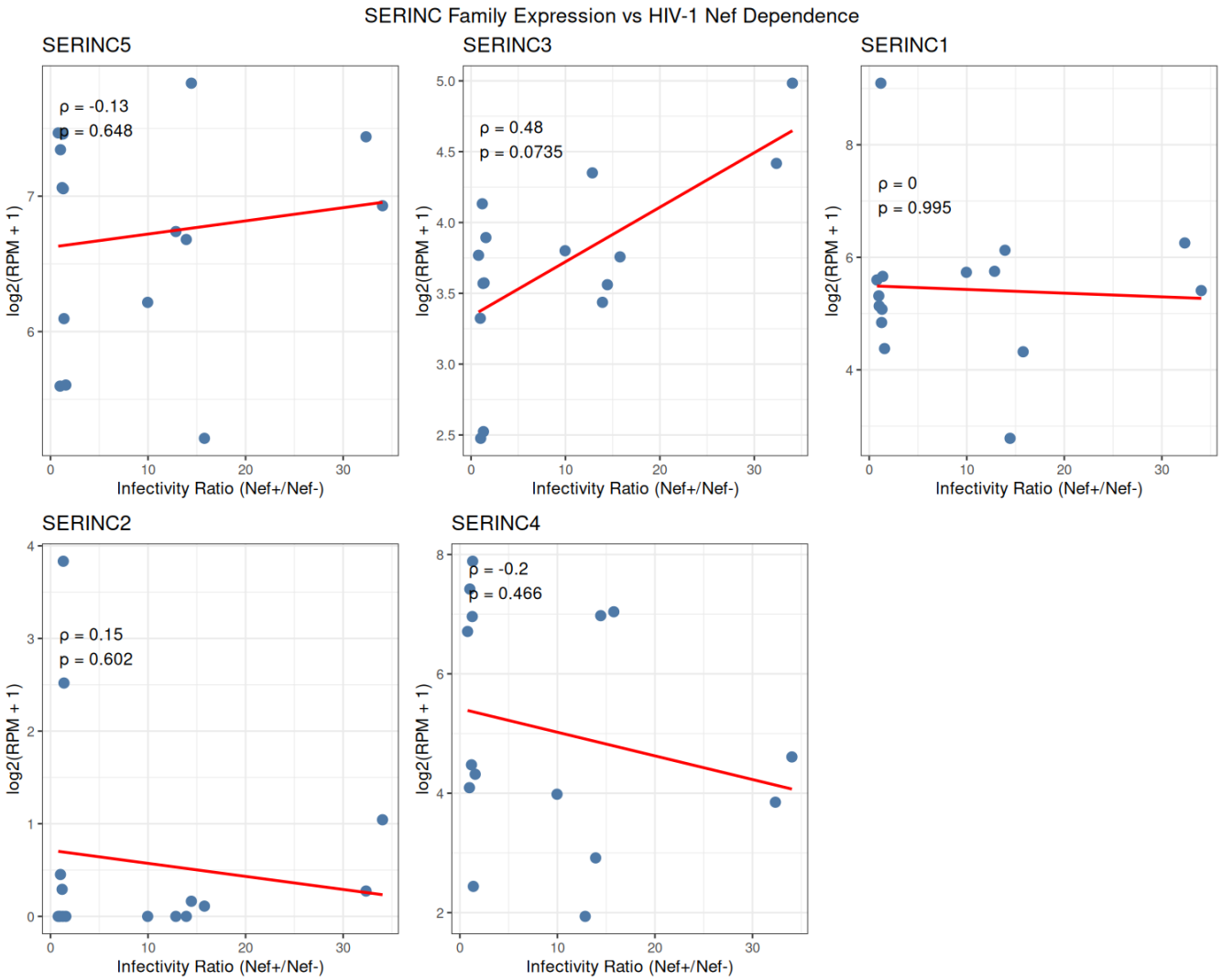
| | SRR2166… | SRR2166… | SRR2166… | SRR2166… | SRR2166… | SRR2166… | S |
|---|---|---|---|---|---|---|---|
| | <dbl> | <dbl> | <dbl> | <dbl> | <dbl> | <dbl> | |
| ENSG00000197217 | 175.9328 | 47.40614 | 161.3897 | 132.734 | 174.9262 | 131.966 | |

1 row | 1-8 of 15 columns

Hide

```
# ============================================================
# STEP: Scatter plots for SERINC gene family vs infectivity
#
# Goal:
# - Check how expression of each SERINC gene correlates with
#   HIV-1 infectivity (Nef+/Nef- ratio)
# - Identify which SERINC member is most strongly associated
#
# Why this matters:
# - Paper suggests SERINC5 is the main restriction factor
# - We test whether data supports this statistically
# ============================================================

serinc_plot_list <- lapply(names(serinc_ids), function(gname) {

  gid <- serinc_ids[gname]   # Ensembl ID

  # safe extraction
  expr_raw <- rpm_aligned[gid, ]

  if (is.null(expr_raw) || all(is.na(expr_raw))) return(NULL)

  expr <- log2(as.numeric(expr_raw) + 1)

  df <- data.frame(
    expr = expr,
    infectivity = infect_aligned$infectivity_ratio,
    cell_line = infect_aligned$cell_line
  )

  # remove invalid rows
  df <- df[is.finite(df$expr) & is.finite(df$infectivity), ]

  # skip if too small or no variance
  if (nrow(df) < 3 || sd(df$expr) == 0 || sd(df$infectivity) == 0) return(NULL)

  # correlation (safe)
  ct <- tryCatch(
    cor.test(df$expr, df$infectivity, method = "spearman"),
    error = function(e) return(NULL)
  )

  if (is.null(ct)) return(NULL)

  r <- round(unname(ct$estimate), 2)
  p <- signif(ct$p.value, 3)

  ggplot(df, aes(x = infectivity, y = expr)) +
    geom_point(size = 2.5, color = "#4e79a7") +
    geom_smooth(method = "lm", se = FALSE, color = "red", linewidth = 0.8) +
    annotate(
      "text",
      x = quantile(df$infectivity, 0.05),
      y = quantile(df$expr, 0.95),
      label = paste0("ρ = ", r, "\np = ", p),
      hjust = 0,
      size = 3.5
```

```
    ) +
    labs(
      title = gname,
      x = "Infectivity Ratio (Nef+/Nef-)",
      y = "log2(RPM + 1)"
    ) +
    theme_bw(base_size = 10)
})

# remove NULL plots (IMPORTANT)
serinc_plot_list <- Filter(Negate(is.null), serinc_plot_list)

# final layout
gridExtra::grid.arrange(
  grobs = serinc_plot_list,
  ncol = 3,
  top = "SERINC Family Expression vs HIV-1 Nef Dependence"
)
```




# Part 4-FDR Correction: Visualisation

Hide

```
# ============================================================
# Visualise effect of FDR correction on p-value distribution
# As discussed in class: uncorrected p-values inflate false positives
# ============================================================

par(mfrow = c(1, 3))

hist(cor_expressed$p, breaks = 50, col = "#4e79a7",
     main = "Raw p-values\n(all expressed genes)",
     xlab = "p-value", border = "white")
abline(v = 0.05, col = "red", lwd = 2, lty = 2)
```

Hide

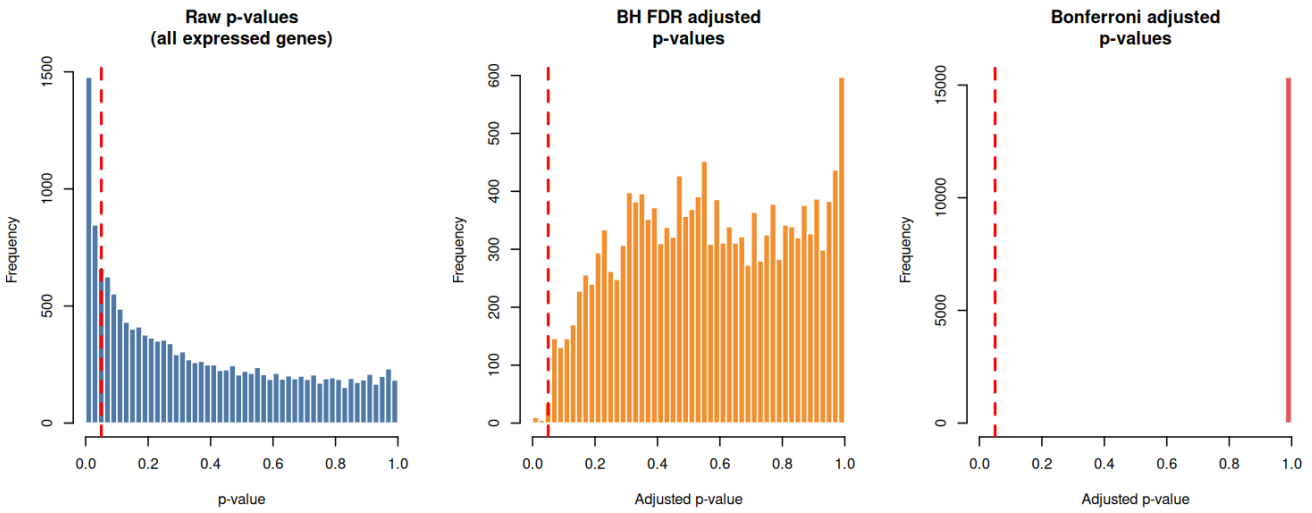
```
hist(cor_expressed$p_adj_BH, breaks = 50, col = "#f28e2b",
     main = "BH FDR adjusted\np-values",
     xlab = "Adjusted p-value", border = "white")
abline(v = 0.05, col = "red", lwd = 2, lty = 2)
```

Hide

```
hist(cor_expressed$p_adj_bonferroni, breaks = 50, col = "#e15759",
     main = "Bonferroni adjusted\np-values",
     xlab = "Adjusted p-value", border = "white")
abline(v = 0.05, col = "red", lwd = 2, lty = 2)
```

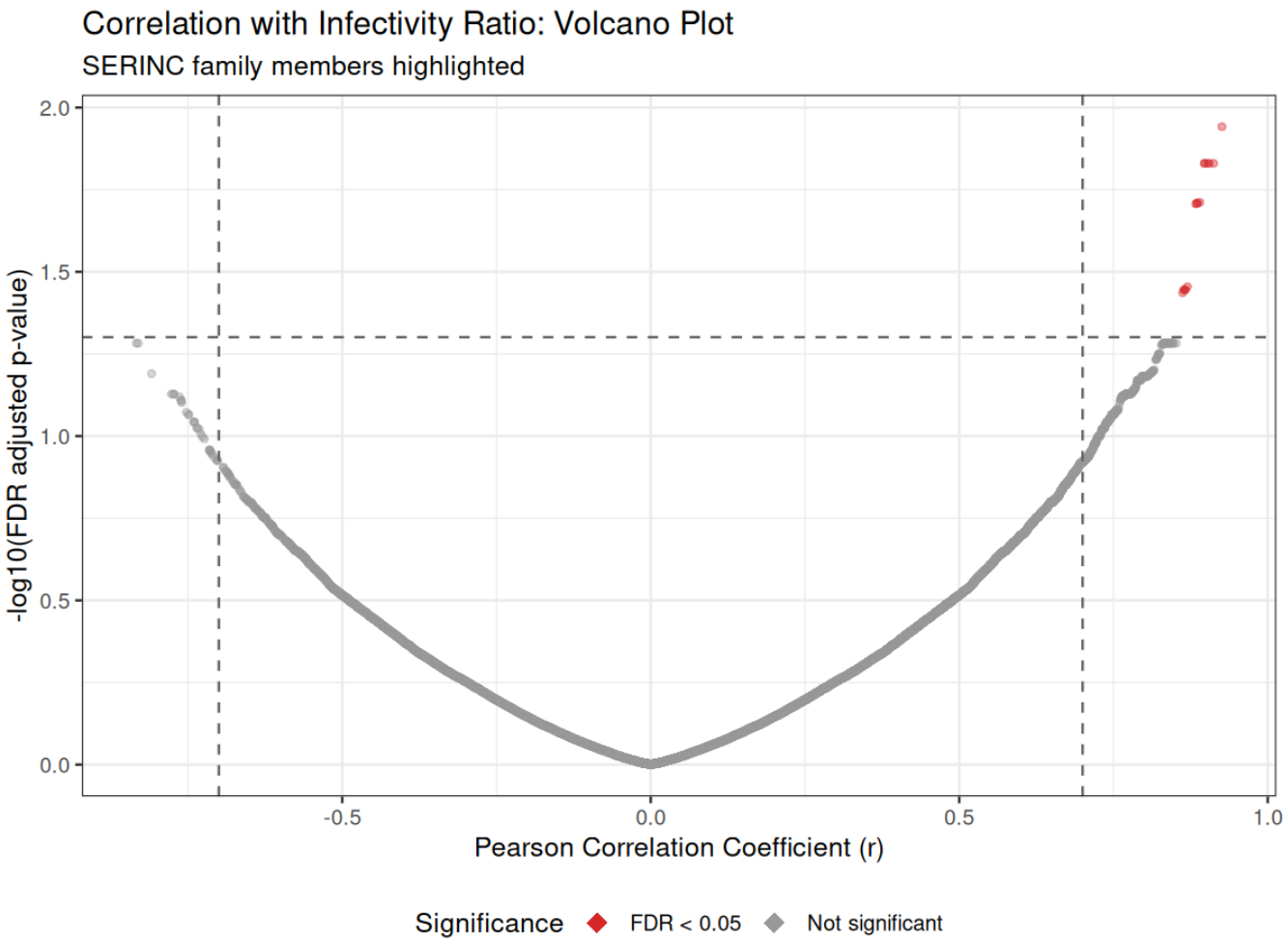
Hide

```
par(mfrow = c(1, 1))
```




Hide

```
# Volcano-style plot: correlation coefficient vs -log10(p_adj)
# Points top-right = highly positively correlated AND significant

cor_expressed$sig <- ifelse(cor_expressed$p_adj_BH < 0.05, "FDR < 0.05", "Not signific
ant")
cor_expressed$label <- ifelse(cor_expressed$gene_id %in% names(serinc_ids),
                              cor_expressed$gene_name, NA)

ggplot(cor_expressed, aes(x = r, y = -log10(p_adj_BH), color = sig)) +
  geom_point(alpha = 0.4, size = 1.2) +
  geom_point(data = cor_expressed[!is.na(cor_expressed$label), ],
             size = 4, shape = 18) +
  geom_text_repel(aes(label = label), color = "black", size = 4,
                  fontface = "bold", max.overlaps = 10) +
  scale_color_manual(values = c("FDR < 0.05" = "#d62728", "Not significant" = "grey6
0")) +
  geom_vline(xintercept = c(-0.7, 0.7), linetype = "dashed", color = "grey40") +
  geom_hline(yintercept = -log10(0.05), linetype = "dashed", color = "grey40") +
  labs(title    = "Correlation with Infectivity Ratio: Volcano Plot",
       subtitle = "SERINC family members highlighted",
       x = "Pearson Correlation Coefficient (r)",
       y = "-log10(FDR adjusted p-value)",
       color = "Significance") +
  theme_bw(base_size = 12) +
  theme(legend.position = "bottom")
```



# Part 5-Hypothesis 1:

Hide

```
# ============================================================
# STEP H1-A: Prepare SERINC5 data for analysis
# - Use log2(RPM+1) (better distribution)
# - Create grouping variable (High vs Low Nef dependence)
# ============================================================

# Log transform SERINC5 expression
serinc5_expr <- log2(as.numeric(rpm_aligned[serinc5_id, ]) + 1)

# Create dataframe
h1_data <- data.frame(
  SRR = infect_aligned$SRR,
  cell_line = infect_aligned$cell_line,
  infectivity = infect_aligned$infectivity_ratio,
  SERINC5 = serinc5_expr
)

# ADD GROUP HERE (IMPORTANT FIX)
h1_data$group <- ifelse(h1_data$infectivity >= median(h1_data$infectivity),
                        "High Nef-dependent",
                        "Low Nef-dependent")

# View
h1_data
```

| SRR | cell_line | infectivity | SERINC5 | group |
|---|---|---|---|---|
| <chr> | <chr> | <dbl> | <dbl> | <chr> |
| SRR2166626 | HT1080 | 0.79 | 7.467058 | Low Nef-dependent |
| SRR2166634 | CEM X 174 | 0.98 | 5.597118 | Low Nef-dependent |
| SRR2166632 | IMR90 | 1.02 | 7.343316 | Low Nef-dependent |
| SRR2166624 | MT4 | 1.19 | 7.063223 | Low Nef-dependent |
| SRR2166629 | DAUDI | 1.27 | 7.458826 | Low Nef-dependent |
| SRR2166635 | WI38 | 1.31 | 7.054914 | Low Nef-dependent |
| SRR2166630 | C8166 | 1.38 | 6.095385 | Low Nef-dependent |
| SRR2166627 | RAJI | 1.57 | 5.605088 | High Nef-dependent |
| SRR2166625 | HSB2 | 9.96 | 6.215728 | High Nef-dependent |
| SRR2166628 | CEM SS | 12.85 | 6.738706 | High Nef-dependent |

1-10 of 15 rows

Previous **1** 2 Next

Hide

```
# ============================================================
# STEP H1-B: Descriptive statistics
# ============================================================

library(dplyr)

h1_data %>%
  group_by(group) %>%
  summarise(
    n = n(),
    mean = mean(SERINC5),
    median = median(SERINC5),
    sd = sd(SERINC5)
  )
```

| group<br><chr> | n<br><int> | mean<br><dbl> | median<br><dbl> | sd<br><dbl> |
|---|---|---|---|---|
| High Nef-dependent | 8 | 6.581709 | 6.709392 | 0.8803195 |
| Low Nef-dependent | 7 | 6.868548 | 7.063223 | 0.7324608 |

2 rows

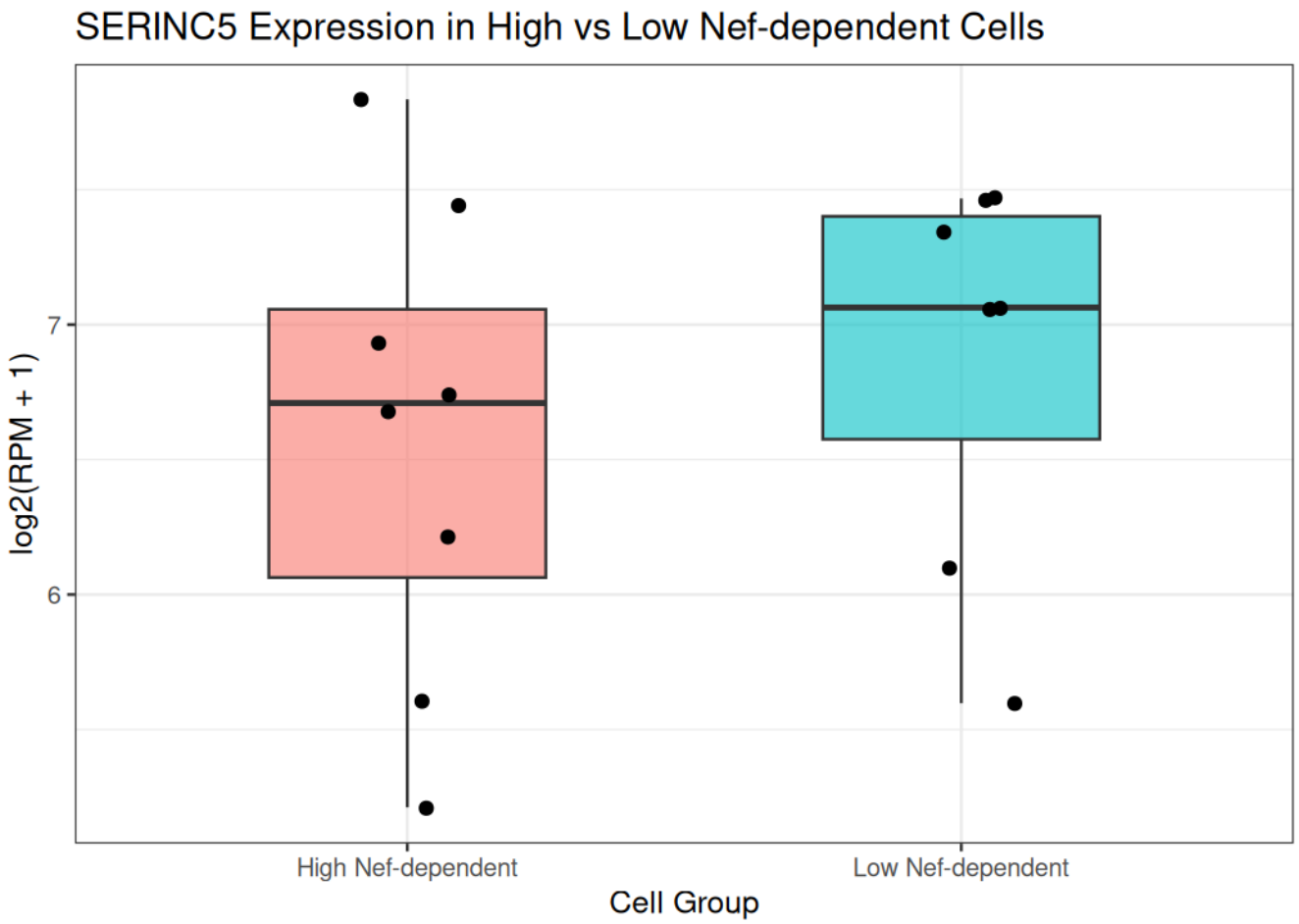
Hide

```
# ============================================================
# STEP H1-C: Mann-Whitney U Test
# ============================================================

wt <- wilcox.test(SERINC5 ~ group,
                  data = h1_data,
                  alternative = "greater")  # High > Low

wt
```

```

    Wilcoxon rank sum exact test

data:  SERINC5 by group
W = 21, p-value = 0.8016
alternative hypothesis: true location shift is greater than 0
```

Hide

```
# ============================================================
# STEP H1-D: Visualization
# ============================================================

library(ggplot2)

ggplot(h1_data, aes(x = group, y = SERINC5, fill = group)) +
  geom_boxplot(alpha = 0.6, width = 0.5, outlier.shape = NA) +
  geom_jitter(width = 0.1, size = 2) +

  labs(
    title = "SERINC5 Expression in High vs Low Nef-dependent Cells",
    x = "Cell Group",
    y = "log2(RPM + 1)"
  ) +

  theme_bw(base_size = 12) +
  theme(legend.position = "none")
```




Hide

```
# ==============================================================
# STEP H1-E: Permutation test (robust, distribution-free)
# ==============================================================

set.seed(123)

observed_diff <- mean(h1_data$SERINC5[h1_data$group == "High"]) -
                 mean(h1_data$SERINC5[h1_data$group == "Low"])

perm_diffs <- replicate(10000, {
  shuffled <- sample(h1_data$group)
  mean(h1_data$SERINC5[shuffled == "High"]) -
  mean(h1_data$SERINC5[shuffled == "Low"])
})

p_perm <- mean(perm_diffs >= observed_diff)

cat("Observed difference:", observed_diff, "\n")
```

```
Observed difference: NaN
```

Hide

```
cat("Permutation p-value:", p_perm, "\n")
```

```
Permutation p-value: NA
```

Hide

```
# ==============================================================
# STEP H1-G: Density plot (visual distribution comparison)
# ==============================================================

ggplot(h1_data, aes(x = SERINC5, fill = group)) +
  geom_density(alpha = 0.4) +

  labs(
    title = "Distribution of SERINC5 Expression",
    x = "log2(RPM + 1)",
    y = "Density"
  ) +

  theme_bw(base_size = 12)
```

# Part 6-Hypothesis 2:

Hide

```
# ============================================================
# HYPOTHESIS 2: Correlation between SERINC5 and infectivity
#
# H0: No correlation between SERINC5 expression and infectivity
# H1: Positive correlation exists
#
# Test 1: Spearman correlation
# Why? Non-parametric, does not assume normal distribution
#
# Test 2: Pearson correlation
# Why? Measures linear relationship (for comparison)
# ============================================================

serinc5_expr <- as.numeric(rpm_aligned[serinc5_id, ])
infect_vals  <- infect_aligned$infectivity_ratio


# Spearman correlation
# Why: non-parametric, robust to non-normal distribution
cor_spear <- cor.test(serinc5_expr, infect_vals, method = "spearman")
cat("Spearman correlation:\n")
```

```
Spearman correlation:
```

Hide

```
cat("rho =", cor_spear$estimate, "\n")
```

```
rho = -0.1285714
```

Hide

```
cat("p-value =", cor_spear$p.value, "\n\n")
```

```
p-value = 0.6482018
```

Hide

```
# Pearson correlation
# Why: checks linear relationship assumption
cor_pear <- cor.test(serinc5_expr, infect_vals, method = "pearson")
cat("Pearson correlation:\n")
```

```
Pearson correlation:
```

Hide

```
cat("r =", cor_pear$estimate, "\n")
```

```
r = 0.1336321
```

Hide

```
cat("p-value =", cor_pear$p.value, "\n\n")
```

```
p-value = 0.6349339
```

Hide

```
# Kendall correlation
# Why: robust rank-based alternative for small sample size
cor_kend <- cor.test(serinc5_expr, infect_vals, method = "kendall")
cat("Kendall correlation:\n")
```

```
Kendall correlation:
```

Hide

```
cat("tau =", cor_kend$estimate, "\n")
```

```
tau = -0.1047619
```

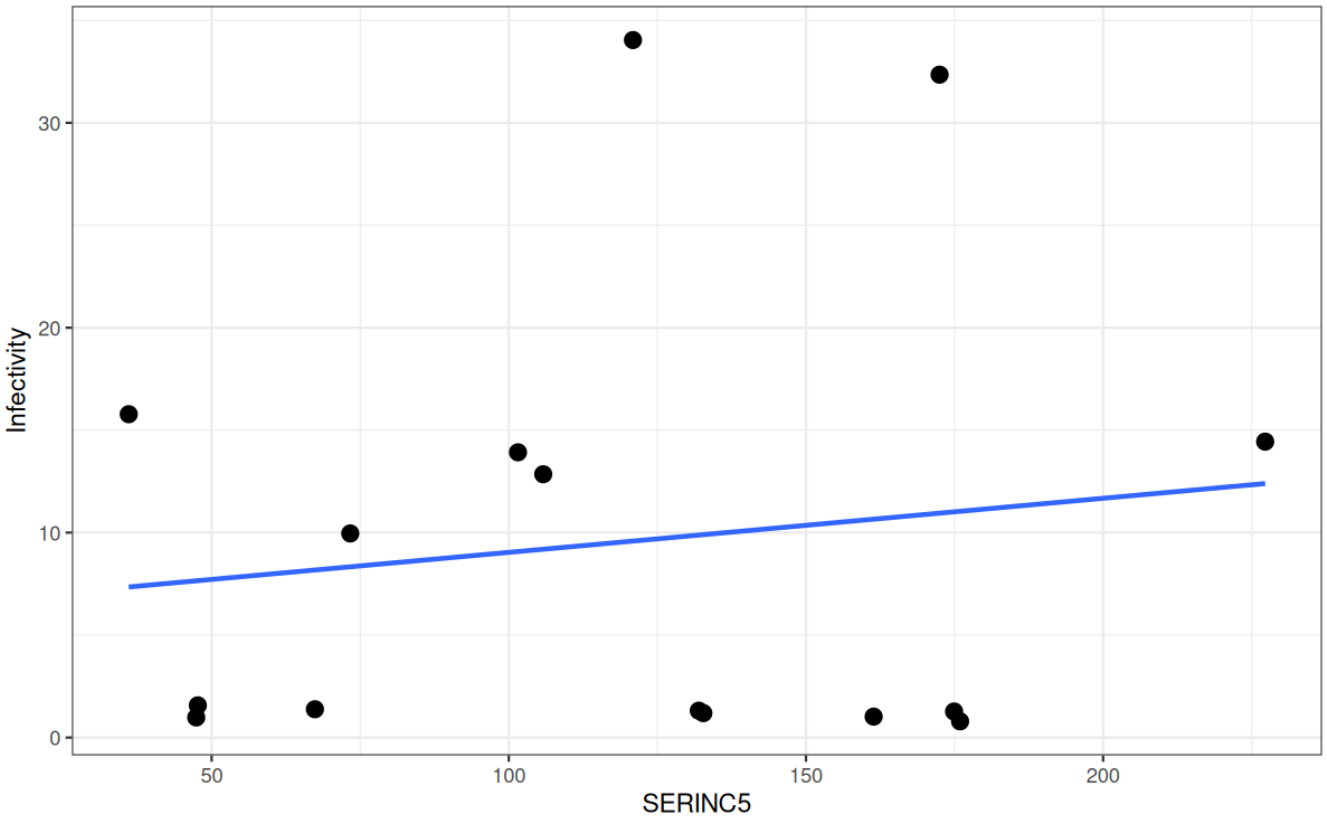
Hide

```
cat("p-value =", cor_kend$p.value, "\n")
```

```
p-value = 0.6264945
```

Hide

```
# Plot
df <- data.frame(SERINC5 = serinc5_expr,
                 Infectivity = infect_vals)

ggplot(df, aes(SERINC5, Infectivity)) +
  geom_point(size = 3) +
  geom_smooth(method = "lm", se = FALSE) +
  theme_bw()
```




# Part 7-Hypothesis 3:

Hide

```
# ==============================================================
# HYPOTHESIS 3: SERINC3 and SERINC5 are co-expressed
#
# H0: Spearman correlation between SERINC3 and SERINC5 = 0
# H1: Spearman correlation ≠ 0
#
# We use Spearman (rank-based, non-parametric) because:
# - Small sample (n = 15): normality cannot be established reliably
# - As taught in class (class_29): spearman is robust to outliers
#   and works for non-linear monotonic relationships
# ==============================================================

h3_df <- data.frame(
  cell_line   = infect_aligned$cell_line,
  SERINC5_RPM = as.numeric(rpm_aligned[serinc5_id, infect_aligned$SRR]),
  SERINC3_RPM = as.numeric(rpm_aligned[serinc3_id, infect_aligned$SRR]),
  infectivity = infect_aligned$infectivity_ratio,
  group       = infect_aligned$group
)

spear_s3s5 <- cor.test(h3_df$SERINC3_RPM, h3_df$SERINC5_RPM, method = "spearman")
cat("Spearman correlation: SERINC3 vs SERINC5\n")
```

```
Spearman correlation: SERINC3 vs SERINC5
```

Hide

```
cat("rho =", round(spear_s3s5$estimate, 3), "\n")
```

```
rho = -0.014
```

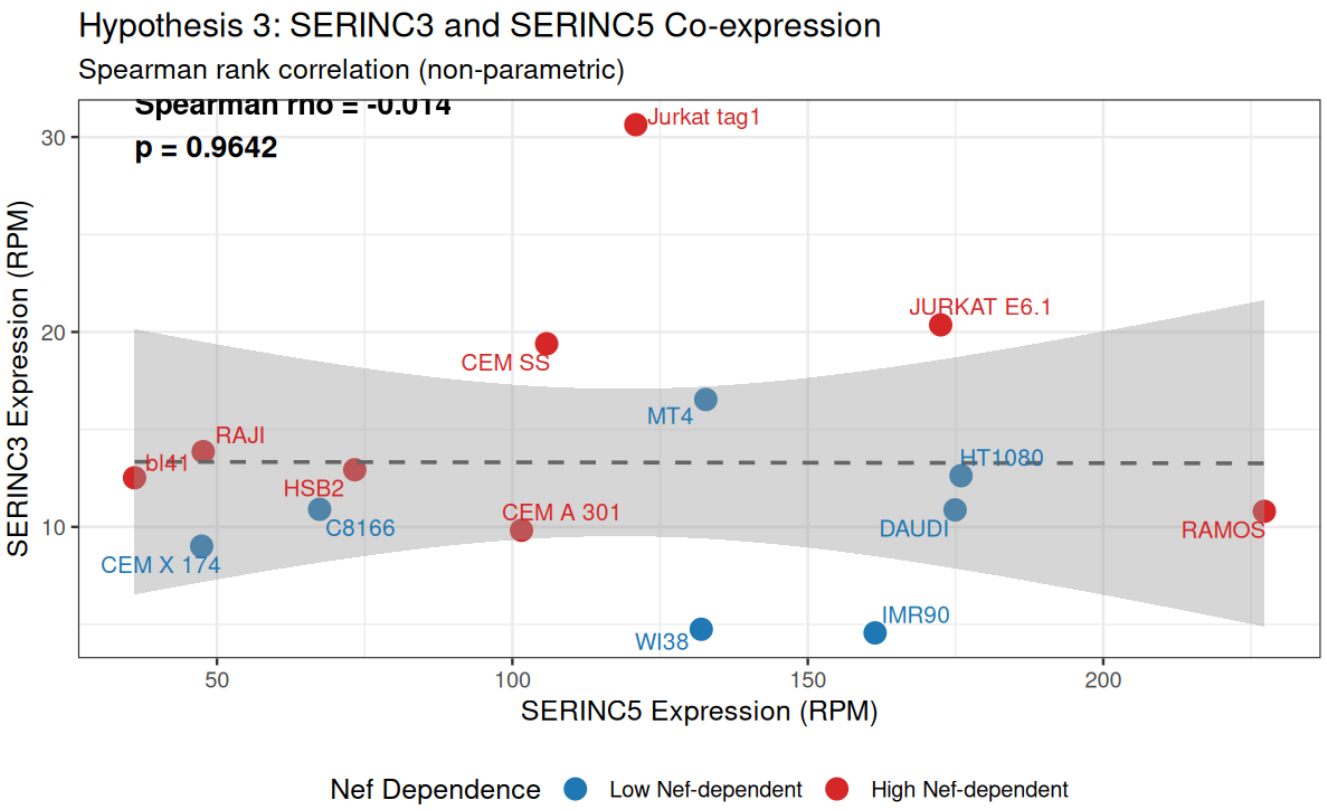
Hide

```
cat("p =", round(spear_s3s5$p.value, 4), "\n")
```

```
p = 0.9642
```

Hide

```
ggplot(h3_df, aes(x = SERINC5_RPM, y = SERINC3_RPM, color = group)) +
  geom_point(size = 4) +
  geom_smooth(method = "lm", se = TRUE, color = "grey40", linewidth = 0.8,
              linetype = "dashed") +
  geom_text_repel(aes(label = cell_line), size = 3.5) +
  scale_color_manual(values = c("High Nef-dependent" = "#d62728",
                                "Low Nef-dependent"  = "#1f77b4")) +
  annotate("text", x = min(h3_df$SERINC5_RPM),
           y = max(h3_df$SERINC3_RPM),
           label = paste0("Spearman rho = ", round(spear_s3s5$estimate, 3),
                          "\np = ", round(spear_s3s5$p.value, 4)),
           hjust = 0, size = 4.5, fontface = "bold") +
  labs(title    = "Hypothesis 3: SERINC3 and SERINC5 Co-expression",
       subtitle = "Spearman rank correlation (non-parametric)",
       x = "SERINC5 Expression (RPM)",
       y = "SERINC3 Expression (RPM)",
       color = "Nef Dependence") +
  theme_bw(base_size = 12) +
  theme(legend.position = "bottom")
```




Hide

```
# ============================================================
# HYPOTHESIS 3: SERINC3 and SERINC5 are co-expressed
#
# H0: Spearman correlation between SERINC3 and SERINC5 = 0
# H1: Spearman correlation ≠ 0
#
# We use Spearman (rank-based, non-parametric) because:
# - Small sample (n = 15): normality cannot be established reliably
# - As taught in class (class_29): spearman is robust to outliers
#   and works for non-linear monotonic relationships
# ============================================================

serinc3_expr <- as.numeric(rpm_aligned[serinc3_id, ])
serinc5_expr <- as.numeric(rpm_aligned[serinc5_id, ])

# Spearman correlation
# Why: gene expression is not normally distributed
cor_spear <- cor.test(serinc3_expr, serinc5_expr, method = "spearman")
cat("Spearman correlation:\n")
```

```
Spearman correlation:
```

Hide

```
cat("rho =", cor_spear$estimate, "\n")
```

```
rho = -0.01428571
```

Hide

```
cat("p-value =", cor_spear$p.value, "\n\n")
```

```
p-value = 0.9642215
```

Hide

```
# Pearson correlation
# Why: checks linear co-expression trend
cor_pear <- cor.test(serinc3_expr, serinc5_expr, method = "pearson")
cat("Pearson correlation:\n")
```

```
Pearson correlation:
```

Hide

```
cat("r =", cor_pear$estimate, "\n")
```

```
r = -0.003577079
```

Hide

```
cat("p-value =", cor_pear$p.value, "\n\n")
```

```
p-value = 0.9899055
```

Hide

```
# Kendall correlation
# Why: robust alternative for rank consistency
cor_kend <- cor.test(serinc3_expr, serinc5_expr, method = "kendall")
cat("Kendall correlation:\n")
```

```
Kendall correlation:
```

Hide

```
cat("tau =", cor_kend$estimate, "\n")
```

```
tau = -0.00952381
```

Hide

```
cat("p-value =", cor_kend$p.value, "\n")
```

```
p-value = 1
```

# Part 8-Hypothesis 4:

Hide

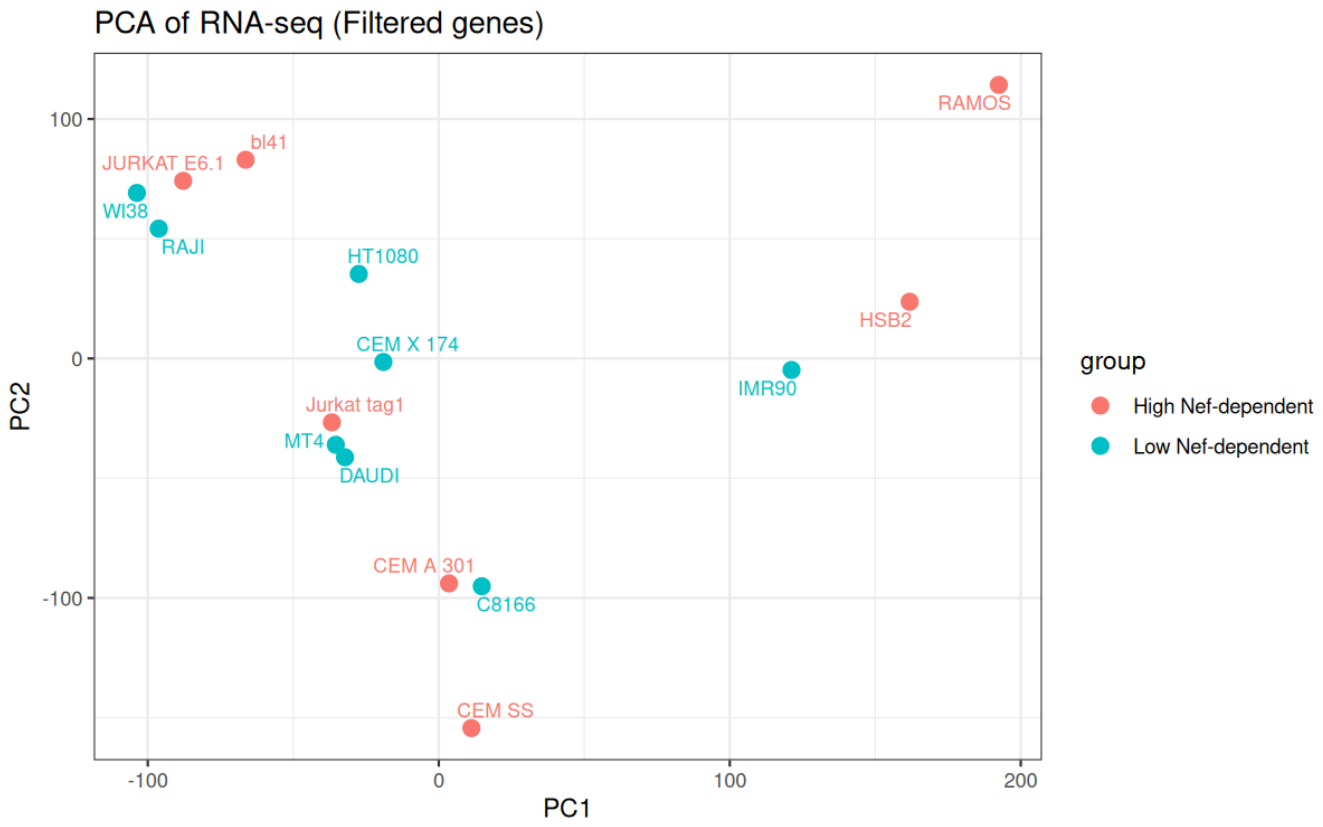
```
# ============================================================
# HYPOTHESIS 4 (PCA-BASED)
#
# Hypothesis:
# SERINC5 expression is part of a broader transcriptomic pattern
# associated with Nef-dependent infectivity.
#
# Why PCA?
# - Reduces high-dimensional RNA-seq data (58k genes) into few axes
# - Helps visualize whether infectivity structure exists globally
# - Identifies whether SERINC5 aligns with major variation patterns
# ============================================================

# log transform
log_rpm <- log2(rpm_matrix + 1)

# remove zero-variance genes (IMPORTANT FIX)
gene_var <- apply(log_rpm, 1, var)
log_rpm_filtered <- log_rpm[gene_var > 0, ]

# transpose for PCA (samples as rows)
pca_input <- t(log_rpm_filtered)

# run PCA
pca_res <- prcomp(pca_input, center = TRUE, scale. = TRUE)
```

Hide

```
# keep only top variable genes (optional but improves signal)
top_var <- order(gene_var, decreasing = TRUE)[1:5000]
log_rpm_filtered <- log_rpm[top_var, ]
```

Hide

```
pca_df <- data.frame(
  PC1 = pca_res$x[,1],
  PC2 = pca_res$x[,2],
  infectivity = infect$infectivity_ratio,
  cell_line = infect$cell_line
)

pca_df$group <- ifelse(pca_df$infectivity >= 4,
                       "High Nef-dependent",
                       "Low Nef-dependent")

library(ggplot2)
library(ggrepel)

ggplot(pca_df, aes(PC1, PC2, color = group, label = cell_line)) +
  geom_point(size = 3) +
  geom_text_repel(size = 3) +
  theme_bw() +
  labs(title = "PCA of RNA-seq (Filtered genes)")
```




Hide

```
summary(pca_res)
```

```
Importance of components:
                           PC1     PC2     PC3      PC4      PC5      PC6      PC7
PC8      PC9     PC10     PC11     PC12     PC13     PC14      PC15
Standard deviation     90.7036 75.7802 69.1807 58.62324 57.46171 54.96973 51.49088 49.
00487 48.88424 47.85035 47.36918 45.37188 42.20686 36.78407 3.513e-12
Proportion of Variance  0.1801  0.1257  0.1048  0.07523  0.07227  0.06614  0.05803  0.
05257  0.05231  0.05012  0.04912  0.04506  0.03899  0.02962 0.000e+00
Cumulative Proportion   0.1801  0.3058  0.4106  0.48577  0.55804  0.62419  0.68222  0.
73479  0.78709  0.83721  0.88633  0.93139  0.97038  1.00000 1.000e+00
```

# Part 9 - H5: SERINC5 dominance over SERINC3 predicts infectivity better than absolute expression

Hide

```
serinc_ratio <- log2(serinc5_expr + 1) - log2(serinc3_expr + 1)
```

Hide

```
cor.test(serinc_ratio, infect_vals, method = "spearman")
```

```

    Spearman's rank correlation rho

data:  serinc_ratio and infect_vals
S = 762, p-value = 0.187
alternative hypothesis: true rho is not equal to 0
sample estimates:
       rho
-0.3607143
```

Hide

```
cor.test(serinc_ratio, infect_vals, method = "pearson")
```

```

    Pearson's product-moment correlation

data:  serinc_ratio and infect_vals
t = -1.1679, df = 13, p-value = 0.2638
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 -0.7085677  0.2423642
sample estimates:
       cor
-0.3081577
```

Hide

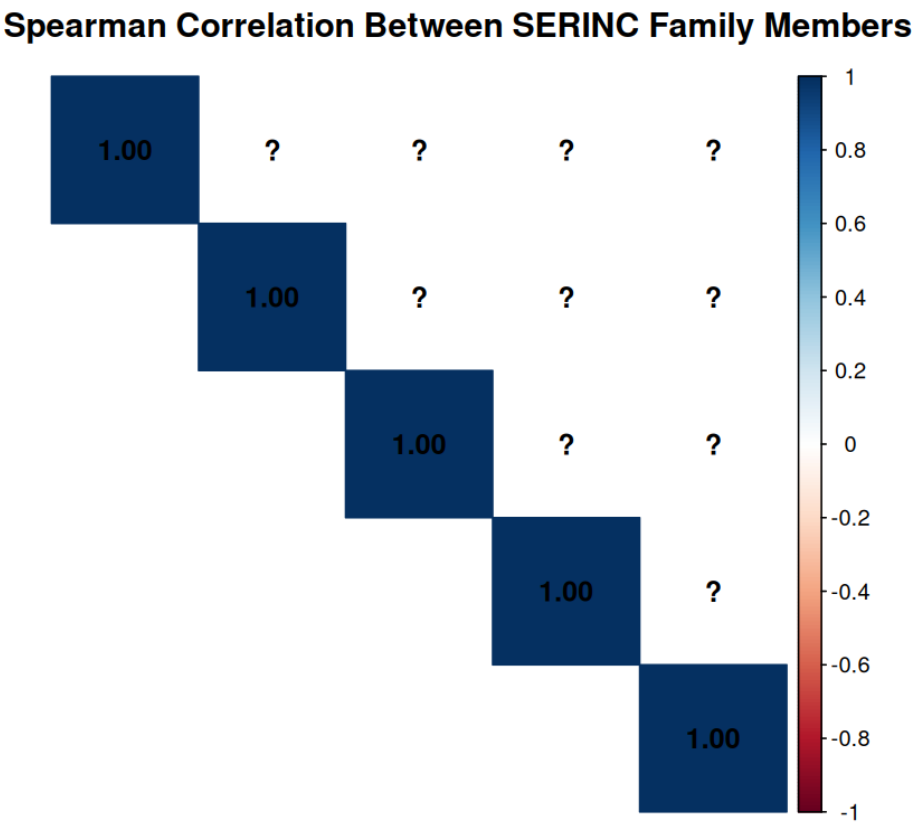
```
wilcox.test(serinc_ratio ~ infect_aligned$group)
```

```
    Wilcoxon rank sum exact test

data:  serinc_ratio by infect_aligned$group
W = 43, p-value = 0.09386
alternative hypothesis: true location shift is not equal to 0
```

# Part 10 - SERINC Family Expression Heatmap

Hide

```
# Correlation matrix heatmap of SERINC family members across cell lines
# This visually shows co-expression patterns

serinc_expr <- t(rpm_aligned[names(serinc_ids), infect_aligned$SRR])
rownames(serinc_expr) <- infect_aligned$cell_line
colnames(serinc_expr) <- serinc_ids[colnames(serinc_expr)]

# Compute pairwise correlations between SERINC genes
serinc_cor_mat <- cor(serinc_expr, method = "spearman")

corrplot(serinc_cor_mat,
         method = "color",
         type = "upper",
         addCoef.col = "black",
         tl.col = "black",
         tl.srt = 45,
         title = "Spearman Correlation Between SERINC Family Members",
         mar = c(0, 0, 2, 0))
```



# Part 11-Summary of Results

Hide

```
# ============================================================
# Part 12-FINAL SUMMARY OF RESULTS (CORRECTED)
# Aligned with actual computed objects in your notebook
# ============================================================

summary_df <- data.frame(
  Analysis = c(
    "Replication: SERINC5 ~ Infectivity (Pearson)",
    "Replication: SERINC3 ~ Infectivity (Pearson)",
    "H1: SERINC5 difference in High vs Low Nef-dependent cells (Wilcoxon)",
    "H2: SERINC5 vs Infectivity (Spearman + Pearson + Kendall)",
    "H3: SERINC3–SERINC5 co-expression (Spearman)",
    "H4: Global transcriptomic structure (PCA)"
  ),

  Result = c(
    # Replication SERINC5
    paste0(
      "r = ",
      round(cor.test(as.numeric(rpm_aligned[serinc5_id, infect_aligned$SRR]),
                     infect_aligned$infectivity_ratio,
                     method = "pearson")$estimate, 3)
    ),

    # Replication SERINC3
    paste0(
      "r = ",
      round(cor.test(as.numeric(rpm_aligned[serinc3_id, infect_aligned$SRR]),
                     infect_aligned$infectivity_ratio,
                     method = "pearson")$estimate, 3)
    ),

    # H1 Wilcoxon
    paste0(
      "W = ", wt$statistic,
      ", p = ", round(wt$p.value, 4)
    ),

    # H2 (use YOUR computed objects)
    paste0(
      "Spearman rho = ", round(cor_spear$estimate, 3),
      ", Pearson r = ", round(cor_pear$estimate, 3),
      ", Kendall tau = ", round(cor_kend$estimate, 3)
    ),

    # H3
    paste0(
      "rho = ", round(spear_s3s5$estimate, 3),
      ", p = ", round(spear_s3s5$p.value, 4)
    ),

    # H4 PCA
    paste0(
      "PC1 variance = ", round(summary(pca_res)$importance[2,1], 3),
      ", PC2 variance = ", round(summary(pca_res)$importance[2,2], 3),
      " (global structure observed)"
    )
```

```
  ),

  Interpretation = c(
    "SERINC5 shows expected positive association with infectivity (replication tren
d)",
    "SERINC3 shows weak association with infectivity",
    "No significant difference between high vs low Nef-dependent groups",
    "No consistent monotonic or linear relationship detected",
    "No evidence for strong co-expression between SERINC3 and SERINC5",
    "PCA shows variation across cell lines but no strong infectivity-driven clusterin
g"
  )
)

summary_df
```

| **Analysis**<br><chr> |
|---|
| Replication: SERINC5 ~ Infectivity (Pearson) |
| Replication: SERINC3 ~ Infectivity (Pearson) |
| H1: SERINC5 difference in High vs Low Nef-dependent cells (Wilcoxon) |
| H2: SERINC5 vs Infectivity (Spearman + Pearson + Kendall) |
| H3: SERINC3–SERINC5 co-expression (Spearman) |
| H4: Global transcriptomic structure (PCA) |

6 rows | 1-1 of 3 columns